



TECNOLÓGICO NACIONAL DE MÉXICO

Instituto Tecnológico Superior de San Martín Texmelucan



DIVISIÓN DE INGENIERÍA EN SISTEMAS COMPUTACIONALES

Simulación

Ejercicios 2

PRESENTA:

Wendoly Isabel Hernández Hernández

PROFESOR:

M.S.C. Ivan Rafael Sánchez Juárez

San Martín Texmelucan, Puebla a 14 de abril de 2025

Contenido

Introducción.....	2
Desarrollo	3
Ejercicio 1. Cuadrados Medios	3
Ejercicio 2. Productos Medios	5
Ejercicio 3. Algoritmo Congruencial Aditivo	7
Ejercicio 4. Algoritmo Blum Blum y Shub	9
Programa 1. Cuadrados Medios	10
Programa 2. Productos Medios	14
Programa 3. Algoritmo Congruencial Lineal.....	19
Firma de Programas (En equipo)	22
Programa. Tombola	23
Conclusión.....	31
Referencias Bibliográficas.....	32

Introducción

En esta actividad se presentan los ejercicios realizados en la unidad correspondiente al tema "Números Pseudoaleatorios", perteneciente a la materia de Simulación. Los números pseudoaleatorios son fundamentales en este campo, ya que permiten imitar el comportamiento de procesos aleatorios en situaciones donde se requiere realizar modelos o experimentos virtuales. Para generar estos números, se utilizaron diferentes métodos, cada uno con sus propias características y aplicaciones. Entre los algoritmos empleados se encuentran los métodos de Cuadrados Medios y Productos Medios, que se basan en operaciones matemáticas sencillas para producir secuencias de números que parecen aleatorios. También se aplicó el método del Multiplicador Constante, una variante de los anteriores, pero con un enfoque más eficiente. Por otro lado, se exploraron técnicas más avanzadas como el Algoritmo Lineal y el Método Congruencial Multiplicativo, los cuales ofrecen mayor control sobre la secuencia generada. Finalmente, se incluyó el Algoritmo Congruencial Aditivo, útil en casos donde se requiere una mayor longitud en las secuencias.

El objetivo de estos ejercicios fue comprender cómo funcionan estos métodos, identificar sus ventajas y limitaciones, y aprender a aplicarlos en problemas de simulación. A través de estos ejercicios, se pudo observar que, aunque los números generados no son verdaderamente aleatorios, su comportamiento es lo suficientemente impredecible para ser útiles en modelos computacionales.

Desarrollo

Ejercicio 1. Cuadrados Medios

Wendoly Kabel Hernandez Hernandez
40-Febrero-2025

2. Números Pseudoaleatorios

Para poder realizar una simulación, es preciso generar una serie de números que sean aleatorios por sí mismos, y que su aleatoriedad se extienda al modelo de simulación.

Cuadrados Medios

Es uno de los métodos para generar números pseudoaleatorios. Requiere un número entero de partida (semilla) con D dígitos, el cual es elevado al cuadrado para seleccionar del resultado los D dígitos del centro.

Ejemplo generar los primeros 15 dígitos:

$X_0 = 5894$	
$X_0 = 5894^2 = 34739236$	$X_4 = 7225^2 = 52200625$
$X_1 = 7392 \quad r_1 = 0.7392$	$X_5 = 1006 \quad r_5 = 0.2006$
$X_1 = 7392^2 = 54641664$	$X_5 = 2006^2 = 4024036$
$X_2 = 6416 \quad r_2 = 0.6416$	$X_6 = 0240 \quad r_6 = 0.0240$
$X_2 = 6416^2 = 41165056$	$X_6 = 0240^2 = 0057600$
$X_3 = 1650 \quad r_3 = 0.1650$	$X_7 = 0576 \quad r_7 = 0.0576$
$X_3 = 1650^2 = 2722500$	$X_7 = 0576^2 = 0331776$
$X_4 = 7225 \quad r_4 = 0.7225$	$X_8 = 3317 \quad r_8 = 0.3317$

Wendoly Isabel Hernandez Hernandez
10-Febrero-2020

$$x_8 = 3317^2 = 11002489$$

$$x_9 = 0024 \quad r_9 = 0.0024$$

$$x_9 = 0024^2 = 000576$$

$$x_{10} = 0005 \quad r_{10} = 0.0005$$

$$x_{10} = 0005^2 = 000025$$

$$x_{11} = 0000 \quad r_{11} = 0.0000$$

$$x_{12} = 0000 = 00000$$

$$x_{12} = 0000 \quad r_{12} = 0.0000$$



$$D = 384107$$

$$x_0 = 384107$$

$$x_0 = 384107^2 = 147538187449$$

$$x_1 = 538187 \quad r_1 = 0.538187$$

$$x_1 = 538187^2 = 289645246969$$

$$x_2 = 645246 \quad r_2 = 0.645246$$

$$x_2 = 645246^2 = 416342400516$$

$$x_3 = 342400 \quad r_3 = 0.342400$$

$$x_3 = 342400^2 = 117237760000$$

$$x_4 = 237760 \quad r_4 = 0.237760$$

$$x_5 = 237760^2 = 56524817600$$

$$x_5 = 524817 \quad r_5 = 0.524817$$

$$x_6 = 524817^2 = 280706053489$$

$$x_6 = 706053 \quad r_6 = 0.706053$$

$$x_7 = 706053^2 = 498510838809$$

$$x_7 = 510838 \quad r_7 = 0.510838$$

FIRST CLASS

Ejercicio 2. Productos Medios

11-Febrero-2025

Productos Medios

- ① Seleccione una semilla (X_0) con D dígitos, $D > 3$
- ② Seleccione una semilla (X_1) con D dígitos, $D > 3$
- ③ Seleccione $y_0 = X_0 * X_1$; sea X_2 los D dígitos del centro y sea $r_1 = 0.D$ dígitos del centro
- ④ Sea $y_i = X_i * X_{i+1}$; sea X_{i+2} los D dígitos del centro $i = 1, 2, 3, \dots, n$.
- ⑤ Repetir el paso 4 hasta obtener los n números pseudo-aleatorios.

Nota 1: Si no es posible obtener los D dígitos del centro, agregue 0's a la izquierda de y_i .

Nota 2: Los dígitos de las semillas deben tener la misma longitud.

Ejemplo: Calcular los NPA con el método de productos medios con $X_0 = 91372$ y $X_1 = 17293$

$$y_1 = X_0 * X_1$$

$$y_1 = 91372 * 17293 = 1580095996$$

$$X_2 = 80095 \quad r_1 = 0.80095$$

$$y_2 = 17293 * 80095 = 01385082835$$

$$X_3 = 85082 \quad r_2 = 0.85082$$

$$y_3 = 80095 * 85082 = 06814642790$$

24-Febrero - 2025

$$x_{62} = (59 \times 149) \cdot 1.256 = 87$$

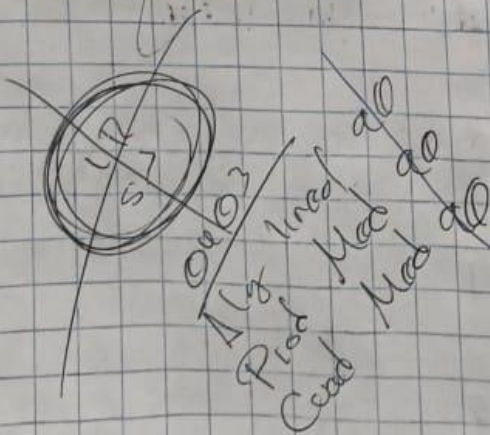
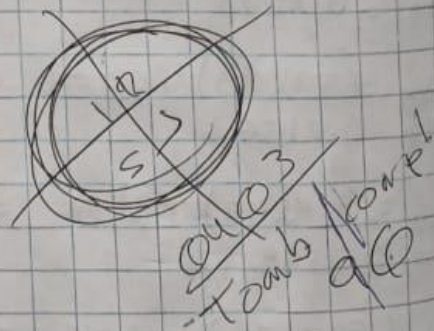
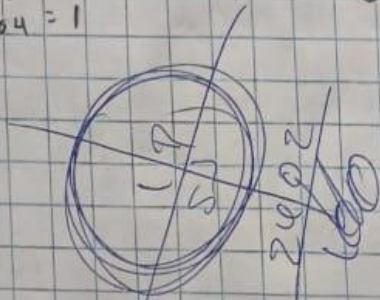
$$r_{62} = 0.3411$$

$$x_{63} = (59 \times 87) \cdot 1.256 = 133$$

$$r_{63} = 0.0509$$

$$x_{64} = (59 \times 13) \cdot 1.256 = 1255$$

$$r_{64} = 1$$



Ejercicio 3. Algoritmo Congruencial Aditivo

Wendoly Isabel Hernandez Hernandez

10-Marzo-2025

Algoritmo congruencial aditivo

Requiere de una secuencia ~~de~~ previa de n números $x_1, x_2, x_3, \dots, x_n$ para generar una nueva secuencia de números $x_{n+1}, x_{n+2}, \dots$

Para el cálculo de los $x_{i+1} = (x_i \cdot r_i) \cdot m$

Para las $r_i = x_i / (m-1)$

Ejemplo / Ejercicio:

Encontrar los primeros 10 NPA con: $x_1 = 78$; $x_2 = 85$; $x_3 = 26$; $x_4 = 04$ y $x_5 = 22$

$m = 100$

$i = 6$

$x_6 =$

$$x_6 = \left(\underset{x_5}{22} \times \underset{x_1}{78} \right) \cdot 100; x_6 = 1716 \cdot 100; x_6 = 16$$

$$r_1 = \frac{16}{99}; r_1 = 0.161616$$

$i = 7$

$$x_7 = \left(\underset{x_6}{16} \times \underset{x_2}{85} \right) \cdot 100; x_7 = 28$$

$$r_2 = \frac{28}{99} = 0.282828$$

$i = 8$

$$x_8 = \left(\underset{x_7}{28} \times \underset{x_3}{26} \right) \cdot 100; x_8 = 28$$

$$r_3 = \frac{28}{99} = 0.282828$$

$i = 9$

$$x_9 = \left(\underset{x_8}{28} \times \underset{x_4}{4} \right) \cdot 100; x_9 = 12$$

$$r_4 = \frac{12}{99} = 0.121212$$

Wendy Isabel Hernandez Hernandez

10-March-2025

$$i=10 \quad \begin{matrix} x_4 & x_5 \\ x_{10} = (12 \times 22) \cdot 100 = 64 \\ r_5 = \frac{64}{99} = 0.646464 \end{matrix}$$

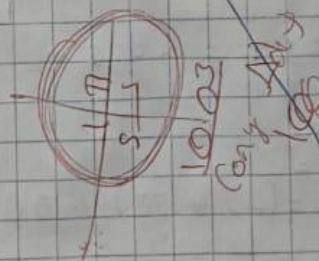
$$x_{18} = (64 \times 16) \cdot 100 = 24 \\ r_{13} = 0.242424$$

$$i=11 \quad \begin{matrix} x_{10} & x_6 \\ x_{11} = (64 \times 16) \cdot 100 = 24 \\ r_6 = 0.242424 \end{matrix}$$

$$x_{19} = (24 \times 92) \cdot 100 = 8 \\ r_{14} = 0.080808$$

$$x_{20} = (8 \times 88) \cdot 100 = 4 \\ r_{15} = 0.040404$$

$$i=12 \quad \begin{matrix} x_{12} = (24 \times 28) \cdot 100 = 72 \\ r_7 = 0.727272 \end{matrix}$$



$$i=13 \quad \begin{matrix} x_{13} = (72 \times 28) \cdot 100 = 16 \\ r_8 = 0.161616 \end{matrix}$$

$$i=14 \quad \begin{matrix} x_{14} = (16 \times 12) \cdot 100 = 92 \\ r_9 = 0.929292 \end{matrix}$$

$$i=15 \quad \begin{matrix} x_{15} = (92 \times 64) \cdot 100 = 88 \\ r_{10} = 0.888888 \end{matrix}$$

$$i=16 \quad \begin{matrix} x_{16} = (88 \times 24) \cdot 100 = 12 \\ r_{11} = 0.121212 \end{matrix}$$

$$x_{17} = (12 \times 72) \cdot 100 = 64 \\ r_{12} = 0.646464$$

Ejercicio 4. Algoritmo Blum Blum y Shub

Wendoly Isabel Hernandez Hernandez.

Algoritmo Blum Blum y Shub

$$x_{i+1} = (a x_i^2) \cdot / \cdot m$$

$$r_i = x_{i+1} / (m-1)$$

EJERCICIO:

Encontrar NPA con $x_0 = 7883$ y $m = 127$ y $a = 27$

$$x_1 = (27(7883^2)) \cdot / \cdot 127 = 28$$

$$r_1 = 0.2222$$

$$x_2 = (27(28^2)) \cdot / \cdot 127 = 86$$

$$r_2 = 0.6825$$

$$x_3 = (27(86^2)) \cdot / \cdot 127 = 48$$

$$r_3 = 0.38095$$

$$x_4 = (27(48^2)) \cdot / \cdot 127 = 105$$

$$r_4 = 0.8333$$

$$x_5 = (27(105^2)) \cdot / \cdot 127 = 114$$

$$r_5 = 0.9047$$

$$x_6 = (27(114^2)) \cdot / \cdot 127 = 118$$

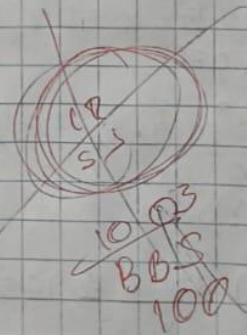
$$r_6 = 0.9365$$

$$x_7 = (27(118^2)) \cdot / \cdot 127 = 28$$

$$r_7 = 0.2222$$

$$x_8 = (27(28^2)) \cdot / \cdot 127 = 86$$

$$r_8 = 0.6825$$



Programa 1. Cuadrados Medios

Este programa genera números pseudoaleatorios a partir de una semilla inicial (un número entero) y genera nuevos números a partir del cuadrado de la semilla, extrayendo los dígitos centrales del resultado. El programa permite al usuario ingresar una semilla y la cantidad de números que desea generar, y luego imprime los resultados en la consola.

Tabla 1. Código “cuadradosMedios”

```
import java.util.Scanner;

public class cuadradosMedios {

    public long x;

    public int D;

    public int n;

    public static void main(String[] args) {

        cuadradosMedios generator = new cuadradosMedios();

        generator.run();

    }

    public void run() {

        Scanner scanner = new Scanner(System.in);

        boolean seguirGenerando = true;

        while (seguirGenerando) {

            System.out.print("Ingrese la semilla (D dígitos, D > 3): ");

            String semilla = scanner.nextLine();
```

```

        while (semilla.length() <= 3) {

            System.out.print("La semilla debe tener más de 3 dígitos. Ingrese
nuevamente: ");

            semilla = scanner.nextLine();

        }

        System.out.print("Ingrese el número de números pseudoaleatorios a generar:
");

        n = Integer.parseInt(scanner.nextLine());

        x = Long.parseLong(semilla);

        D = semilla.length();

        System.out.printf("%-10s %-20s %-20s\n", "x_i+1", "x_i+1 (D dígitos)", "r_i
(0.D dígitos)");

        generarNumerosAleatorios(0);

        System.out.print("¿Desea generar más números con otra semilla? (s/n): ");

        String respuesta = scanner.nextLine();

        seguirGenerando = respuesta.equalsIgnoreCase("s");

    }

    scanner.close();

}

private void generarNumerosAleatorios(int contar) {

    if (contar >= n) {

        return;

    }

```

```

        long y = x * x;

        String yStr = String.valueOf(y);

        int ylongitud = yStr.length();

        int inicioSemilla = (ylongitud - D) / 2;

        int finSemilla = inicioSemilla + D;

        String digitosCentrales;

        if (inicioSemilla < 0) {

            digitosCentrales = String.format("%0" + D + "d", 0);

        } else {

            digitosCentrales = yStr.substring(inicioSemilla, Math.min(finSemilla,
ylongitud));

            while (digitosCentrales.length() < D) {

                digitosCentrales = "0" + digitosCentrales;

            }

        }

        long xSiguiete = Long.parseLong(digitosCentrales);

        double r = (double) xSiguiete / Math.pow(10, D)

        System.out.printf("%-10d %-20s %-20.10f%n", xSiguiete, digitosCentrales, r);

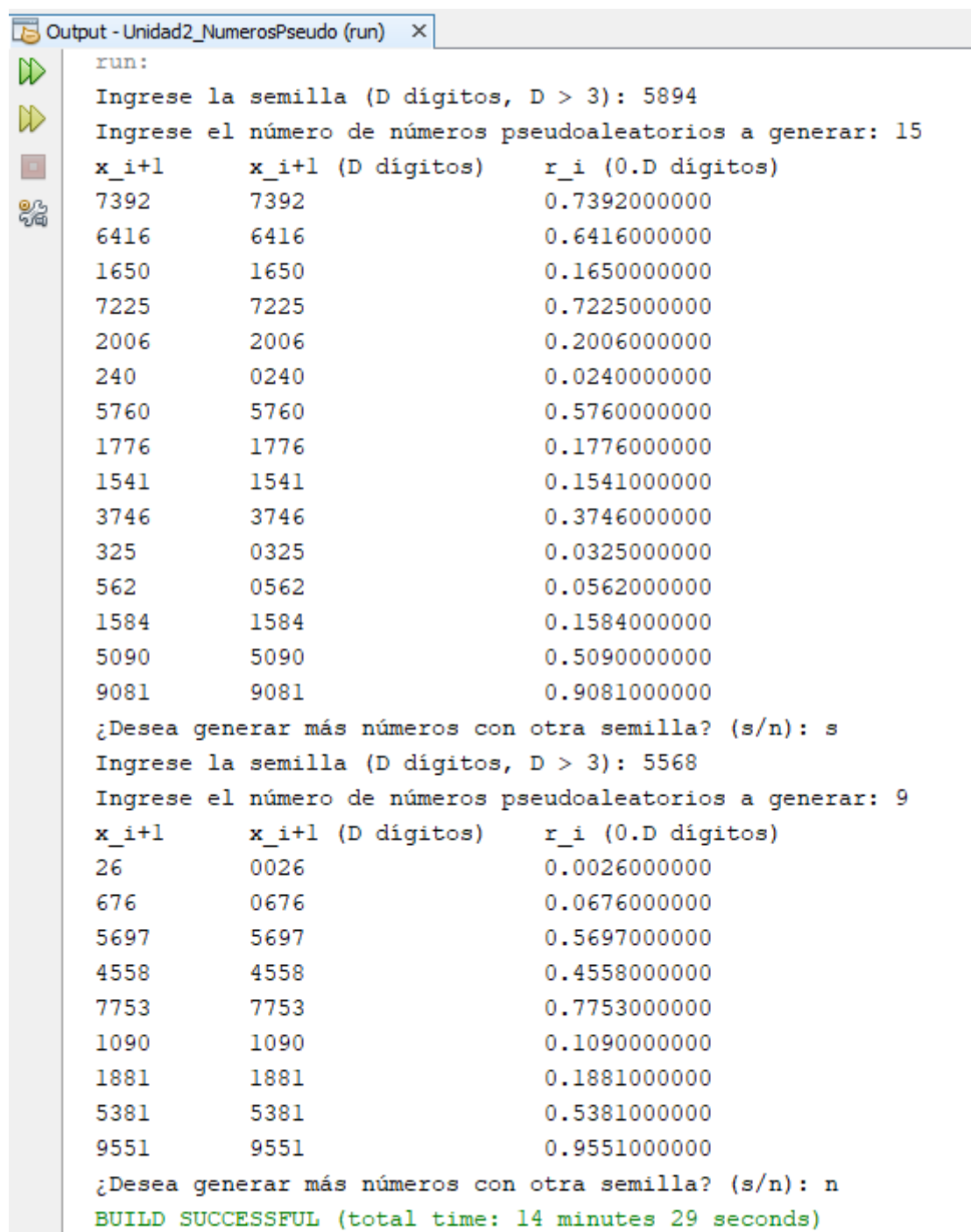
        x = xSiguiete;

        generarNumerosAleatorios(contar + 1);

    }
}

```

Figura 1. Ejecución del código



```
run:
Ingrese la semilla (D dígitos, D > 3): 5894
Ingrese el número de números pseudoaleatorios a generar: 15
x_i+1      x_i+1 (D dígitos)      r_i (0.D dígitos)
7392       7392                   0.7392000000
6416       6416                   0.6416000000
1650       1650                   0.1650000000
7225       7225                   0.7225000000
2006       2006                   0.2006000000
240        0240                   0.0240000000
5760       5760                   0.5760000000
1776       1776                   0.1776000000
1541       1541                   0.1541000000
3746       3746                   0.3746000000
325        0325                   0.0325000000
562        0562                   0.0562000000
1584       1584                   0.1584000000
5090       5090                   0.5090000000
9081       9081                   0.9081000000
¿Desea generar más números con otra semilla? (s/n): s
Ingrese la semilla (D dígitos, D > 3): 5568
Ingrese el número de números pseudoaleatorios a generar: 9
x_i+1      x_i+1 (D dígitos)      r_i (0.D dígitos)
26         0026                   0.0026000000
676        0676                   0.0676000000
5697       5697                   0.5697000000
4558       4558                   0.4558000000
7753       7753                   0.7753000000
1090       1090                   0.1090000000
1881       1881                   0.1881000000
5381       5381                   0.5381000000
9551       9551                   0.9551000000
¿Desea generar más números con otra semilla? (s/n): n
BUILD SUCCESSFUL (total time: 14 minutes 29 seconds)
```

Programa 2. Productos Medios

1. Pide dos semillas numéricas que tengan la misma cantidad de dígitos (más de 3).
2. Calcula el producto de las semillas y extrae los dígitos centrales del resultado.
3. Convierte esos dígitos centrales en un número decimal entre 0 y 1 (esto es el número pseudoaleatorio).
4. Repite el proceso usando las semillas actualizadas en cada iteración.
5. Muestra los resultados en consola y pregunta si deseas repetirlo con otras semillas.

Tabla 2. Código “productosMedios”

```
import java.util.Scanner;

public class productosMedios {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        boolean continuar = true;

        try {

            while (continuar) {

                System.out.print("Ingrese la primera semilla (D dígitos, D > 3): ");

                String semilla1 = scanner.next();

                while (semilla1.length() <= 3) {

                    System.out.print("La semilla debe tener más de 3 dígitos. Ingrese nuevamente: ");

                    semilla1 = scanner.next();

                }

            }

        }

    }

}
```

```

    }

    System.out.print("Ingrese la segunda semilla (D dígitos, D > 3): ");

    String semilla2 = scanner.next();

    while (semilla2.length() <= 3 || semilla1.length() != semilla2.length())
    {

        if (semilla2.length() <= 3) {

            System.out.print("La semilla debe tener más de 3 dígitos. Ingrese
nuevamente: ");

        } else {

            System.out.print("Las semillas deben tener la misma longitud.
Ingrese nuevamente: ");

        }

        semilla2 = scanner.next();

    }

    System.out.print("Ingrese el número de números pseudoaleatorios a
generar: ");

    int n = scanner.nextInt();

    long x0 = Long.parseLong(semilla1);

    long x1 = Long.parseLong(semilla2);

    int D = semilla1.length();

    System.out.println("\nResultados:");

    System.out.printf("%-10s %-20s %-20s%n", "x_i+1", "x_i+1 (D dígitos)",
"r_i (0.D dígitos)");

    for (int i = 0; i < n; i++) {

        long y0 = x0 * x1;

```

```

        String digitosCentralesy0 = getDigitosCentrales(y0, D);

        double r0 = (double) Long.parseLong(digitosCentralesy0) /
Math.pow(10, D);

        System.out.printf("%-10d          %-20s          %-20.10f%n",
Long.parseLong(digitosCentralesy0), digitosCentralesy0, r0);

        long y1 = x1 * x1;

        String digitosCentralesY1 = getDigitosCentrales(y1, D);

        x0 = Long.parseLong(digitosCentralesy0);

        x1 = Long.parseLong(digitosCentralesY1);

    }

    System.out.print("\n¿Desea generar más números con otras semillas?
(s/n): ");

    String respuesta = scanner.next();

    continuar = respuesta.equalsIgnoreCase("s");

}

} catch (Exception e) {

    System.out.println("Error: " + e);

} finally {

    scanner.close();

}

}

private static String getDigitosCentrales(long numero, int D) {

    String numeroStr = String.valueOf(numero);

    int longitud = numeroStr.length();

```

```

        int inicioSemilla = (longitud - D) / 2;

        int finSemilla = inicioSemilla + D;

        if (inicioSemilla < 0) {

            return String.format("%0" + D + "d", 0);

        } else {

            String digitosCentrales = numeroStr.substring(inicioSemilla,
Math.min(finSemilla, longitud));

            while (digitosCentrales.length() < D) {

                digitosCentrales = "0" + digitosCentrales;

            }

            return digitosCentrales;

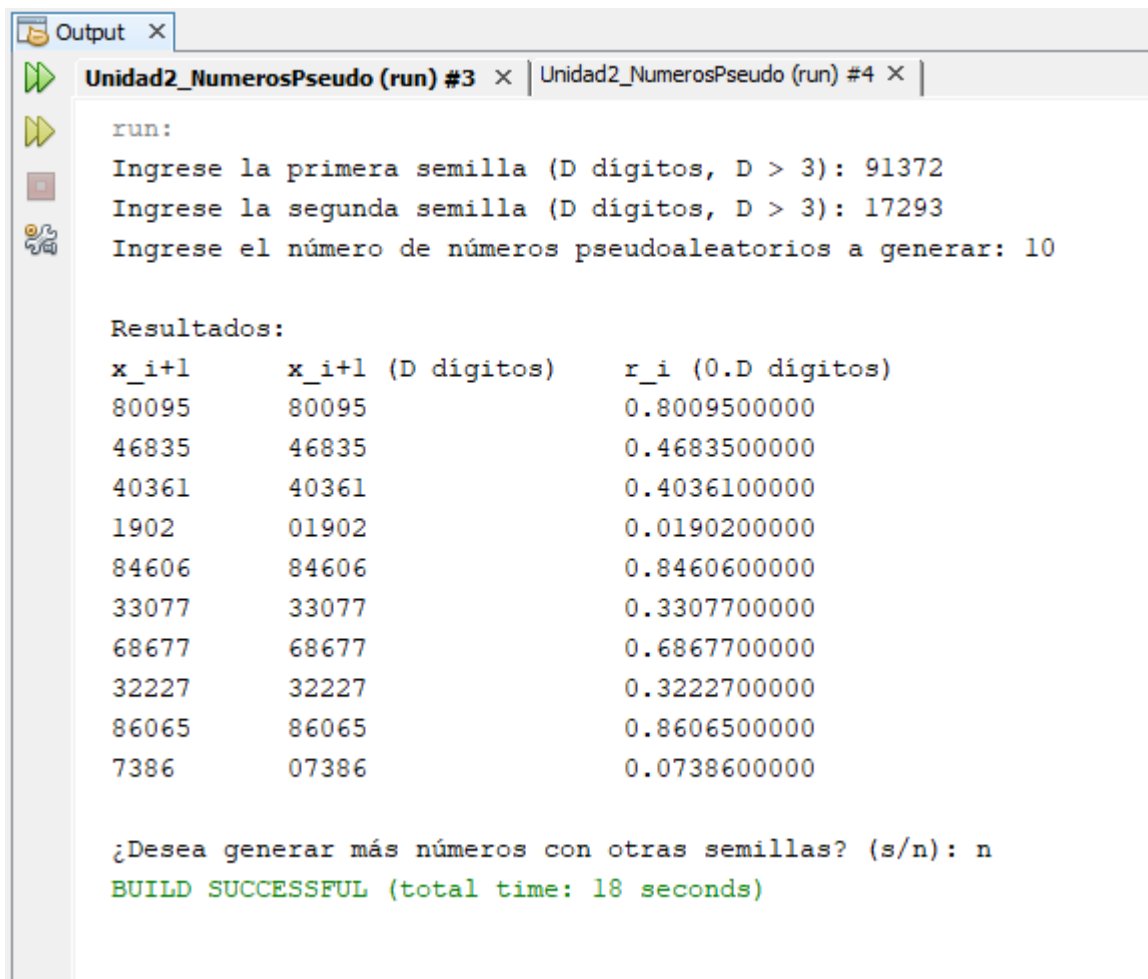
        }

    }

}

```

Figura 2. Ejecución del programa



The screenshot shows a terminal window titled "Output" with two tabs: "Unidad2_NumerosPseudo (run) #3" and "Unidad2_NumerosPseudo (run) #4". The first tab is active and displays the following text:

```
run:
Ingrese la primera semilla (D dígitos, D > 3): 91372
Ingrese la segunda semilla (D dígitos, D > 3): 17293
Ingrese el número de números pseudoaleatorios a generar: 10

Resultados:
x_i+1      x_i+1 (D dígitos)      r_i (0.D dígitos)
80095      80095                      0.8009500000
46835      46835                      0.4683500000
40361      40361                      0.4036100000
1902       01902                      0.0190200000
84606      84606                      0.8460600000
33077      33077                      0.3307700000
68677      68677                      0.6867700000
32227      32227                      0.3222700000
86065      86065                      0.8606500000
7386       07386                      0.0738600000

¿Desea generar más números con otras semillas? (s/n): n
BUILD SUCCESSFUL (total time: 18 seconds)
```

The table of results has three columns: `x_i+1`, `x_i+1 (D dígitos)`, and `r_i (0.D dígitos)`. It lists 10 rows of generated pseudo-random numbers and their corresponding decimal representations.

Programa 3. Algoritmo Congruencial Lineal

Para la generación de números pseudoaleatorios con este algoritmo, pide al usuario:

x_0 → semilla inicial, el punto de partida del algoritmo.

k y g → se usan para calcular los parámetros del algoritmo:

$m = 2^g$ → módulo (el rango máximo de los números generados),

$a = 1 + 4k$ → multiplicador (asegura buena dispersión de los datos).

c = constante aditiva. Debe ser relativamente primo con m para asegurar el máximo periodo (es decir, que no se repitan tan rápido los números generados).

Tabla 1. Código “AlgoritmoCongruencialLineal”

```
import java.util.Scanner;

public class AlgoritmoCongruencialLineal {
    public static void main(String[] args) {
        AlgoritmoCongruencialLineal generator = new AlgoritmoCongruencialLineal();
        generator.run();
    }
    public void run() {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.print("Ingrese el valor de la semilla (x0): ");
            long x0 = scanner.nextLong();

            System.out.print("Ingrese el valor de k (entero): ");
            int k = scanner.nextInt();

            System.out.print("Ingrese el valor de g (entero): ");
            int g = scanner.nextInt();

            long m = (long) Math.pow(2, g);
            long a = 1 + 4 * k;
```

```

        long c;
        do {
            System.out.print("Ingrese el valor de c (debe ser relativamente primo a
m = " + m + "): ");
            c = scanner.nextLong();
            if (gcd(c, m) != 1) {
                System.out.println("El valor de c no es relativamente primo a m.
Intente nuevamente.");
            }
        } while (gcd(c, m) != 1);

        StringBuilder resultados = new StringBuilder();
        resultados.append("Resultados:\n");
        resultados.append(String.format("%-10s %-20s\n", "x_i+1", "r_i+1"));

        long xi = x0;
        for (int i = 0; i < m; i++) {
            long xi1 = (a * xi + c) % m;
            double ri1 = (double) xi1 / (m - 1);

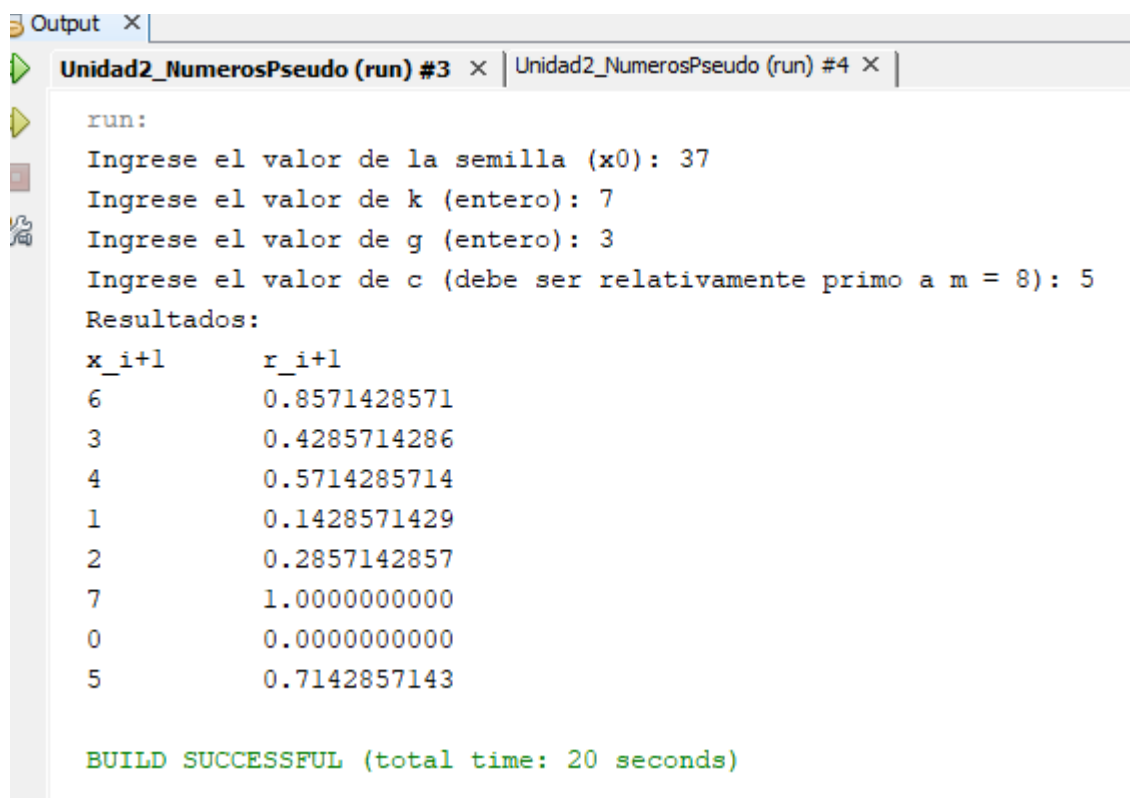
            resultados.append(String.format("%-10d %-20.10f\n", xi1, ri1));
            xi = xi1;
        }
        System.out.println(resultados);

    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
    scanner.close();
}

private long gcd(long a, long b) {
    while (b != 0) {
        long temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
}

```

Figura 3. Ejecución del código

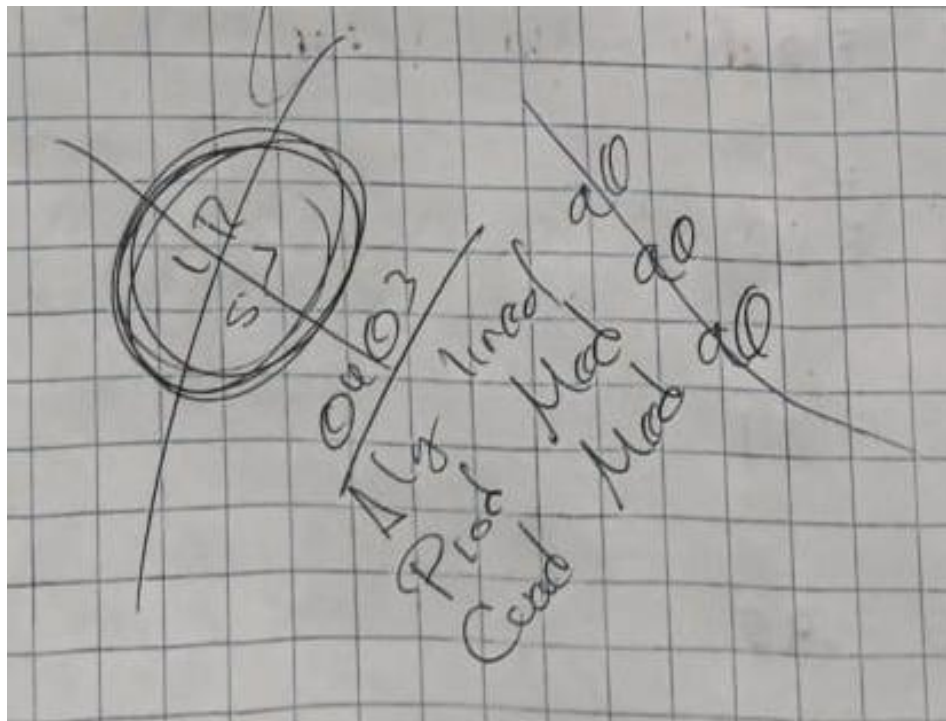


The screenshot shows a code execution environment with two tabs: 'Unidad2_NumerosPseudo (run) #3' and 'Unidad2_NumerosPseudo (run) #4'. The output for the first run is as follows:

```
run:
Ingrese el valor de la semilla (x0): 37
Ingrese el valor de k (entero): 7
Ingrese el valor de g (entero): 3
Ingrese el valor de c (debe ser relativamente primo a m = 8): 5
Resultados:
x_i+1      r_i+1
6          0.8571428571
3          0.4285714286
4          0.5714285714
1          0.1428571429
2          0.2857142857
7          1.0000000000
0          0.0000000000
5          0.7142857143

BUILD SUCCESSFUL (total time: 20 seconds)
```

Firma de Programas (En equipo)



Programa. Tombola

Este programa genera imágenes aleatorias de un conjunto de 30 imágenes disponibles cada que se presiona el botón "GENERAR". Muestra 3 imágenes diferentes cada vez que se presiona el botón, sin repetir hasta que se terminen todas. Cuando ya no hay suficientes imágenes, se reinicia la lista de forma aleatoria.

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Random;
import javax.swing.ImageIcon;

public class fotos extends javax.swing.JFrame {

    private static final int numAlumnos = 30;

    private ArrayList<Integer> numerosDisponibles = new ArrayList<>();

    public fotos() {

        initComponents();

        inicializarLista();

    }

    private void inicializarLista() {

        numerosDisponibles.clear();

        for (int i = 1; i <= numAlumnos; i++) {

            numerosDisponibles.add(i);

        }

        Collections.shuffle(numerosDisponibles);

    }

    @SuppressWarnings("unchecked")

    // <editor-fold defaultstate="collapsed" desc="Generated Code">

    private void initComponents() {
```

```

jPanel1 = new javax.swing.JPanel();

jLabel1 = new javax.swing.JLabel();

jLabel2 = new javax.swing.JLabel();

jLabel3 = new javax.swing.JLabel();

jButton1 = new javax.swing.JButton();

jLabel4 = new javax.swing.JLabel();

setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

jPanel1.setBackground(new java.awt.Color(255, 102, 102));

jButton1.setBackground(new java.awt.Color(255, 51, 102));

jButton1.setFont(new java.awt.Font("Tahoma", 3, 18)); // NOI18N

jButton1.setText("GENERAR");

jButton1.addMouseListener(new java.awt.event.MouseAdapter() {

    public void mouseClicked(java.awt.event.MouseEvent evt) {

        jButton1MouseClicked(evt);

    }

});

jButton1.addActionListener(new java.awt.event.ActionListener() {

    public void actionPerformed(java.awt.event.ActionEvent evt) {

        jButton1ActionPerformed(evt);

    }

});

jLabel4.setFont(new java.awt.Font("Tahoma", 3, 36)); // NOI18N

jLabel4.setForeground(new java.awt.Color(153, 0, 51));

jLabel4.setText("IMAGENES ALEATORIAS");

```

```

        javax.swing.GroupLayout jPanel1Layout = new javax.swing.GroupLayout(jPanel1);

        jPanel1.setLayout(jPanel1Layout);

        jPanel1Layout.setHorizontalGroup(
jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

            .addGroup(jPanel1Layout.createSequentialGroup()

                .addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

                    .addGroup(jPanel1Layout.createSequentialGroup()

                        .addGroup(jPanel1Layout.createSequentialGroup()

                            .addGap(66, 66, 66)

                            .addComponent(jLabel1, javax.swing.GroupLayout.PREFERRED_SIZE,
250, javax.swing.GroupLayout.PREFERRED_SIZE)

                            .addGap(18, 18, 18)

                            .addComponent(jLabel2, javax.swing.GroupLayout.PREFERRED_SIZE,
250, javax.swing.GroupLayout.PREFERRED_SIZE)

                            .addGap(18, 18, 18)

                            .addComponent(jLabel3, javax.swing.GroupLayout.PREFERRED_SIZE,
250, javax.swing.GroupLayout.PREFERRED_SIZE))

                        .addGroup(jPanel1Layout.createSequentialGroup()

                            .addGroup(jPanel1Layout.createSequentialGroup()

                                .addGap(238, 238, 238)

                                .addComponent(jLabel14))

                            .addGroup(jPanel1Layout.createSequentialGroup()

                                .addGap(393, 393, 393)

                                .addComponent(jButton1)))

                        .addContainerGap(75, Short.MAX_VALUE))

                    .addGroup(jPanel1Layout.createSequentialGroup()

                        .addGroup(jPanel1Layout.createSequentialGroup()

                            .addGap(25, 25, 25)

```

```

        .addComponent(jLabel4)

        .addGap(73, 73, 73)

.addGroup(jPanel1Layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)

        .addComponent(jLabel1, javax.swing.GroupLayout.PREFERRED_SIZE, 250,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addComponent(jLabel3, javax.swing.GroupLayout.PREFERRED_SIZE, 250,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addComponent(jLabel2, javax.swing.GroupLayout.PREFERRED_SIZE, 250,
javax.swing.GroupLayout.PREFERRED_SIZE))

        .addGap(27, 27, 27)

        .addComponent(jButton1, javax.swing.GroupLayout.PREFERRED_SIZE, 46,
javax.swing.GroupLayout.PREFERRED_SIZE)

        .addContainerGap(84, Short.MAX_VALUE))

);

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);

layout.setHorizontalGroup(

    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addComponent(jPanel1, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)

);

layout.setVerticalGroup(

    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)

        .addComponent(jPanel1, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)

);

pack();

} // </editor-fold>

```

```

private void jButton1MouseClicked(java.awt.event.MouseEvent evt) {

    if (numerosDisponibles.size() < 3) {

        inicializarLista();

    }

    int persona1 = numerosDisponibles.remove(0);

    int persona2 = numerosDisponibles.remove(0);

    int persona3 = numerosDisponibles.remove(0);

    mostrarImagen(jLabel1, persona1);

    mostrarImagen(jLabel2, persona2);

    mostrarImagen(jLabel3, persona3);

    System.out.println("Imágenes generadas: " + persona1 + ", " + persona2 + ", " +
persona3);

}

private void mostrarImagen(javax.swing.JLabel label, int numero) {

    String rutaImagen = "/SIMU/" + numero + ".jpeg";

    try {

        ImageIcon imagen = new ImageIcon(getClass().getResource(rutaImagen));

        label.setIcon(imagen);

    } catch (Exception e) {

        System.err.println("Error al cargar la imagen: " + rutaImagen);

        e.printStackTrace();

    }

}

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {

    // TODO add your handling code here:

}

```

```

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {

    /* Set the Nimbus look and feel */

    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code
(optional) ">

    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default
look and feel.

        *           For           details           see
http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html

        */

    try {

        for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {

            if ("Nimbus".equals(info.getName())) {

                javax.swing.UIManager.setLookAndFeel(info.getClassName());

                break;

            }

        }

    } catch (ClassNotFoundException ex) {

java.util.logging.Logger.getLogger(fotos.class.getName()).log(java.util.logging.Level.S
EVERE, null, ex);

    } catch (InstantiationException ex) {

java.util.logging.Logger.getLogger(fotos.class.getName()).log(java.util.logging.Level.S
EVERE, null, ex);

    } catch (IllegalAccessException ex) {

```

```

java.util.logging.Logger.getLogger(fotos.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);

        } catch (javax.swing.UnsupportedLookAndFeelException ex) {

java.util.logging.Logger.getLogger(fotos.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);

        }

        //</editor-fold>

        /* Create and display the form */

        java.awt.EventQueue.invokeLater(new Runnable() {

            public void run() {

                new fotos().setVisible(true);

            }

        });

    }

    // Variables declaration - do not modify

    private javax.swing.JButton jButton1;

    private javax.swing.JLabel jLabel1;

    private javax.swing.JLabel jLabel2;

    private javax.swing.JLabel jLabel3;

    private javax.swing.JLabel jLabel4;

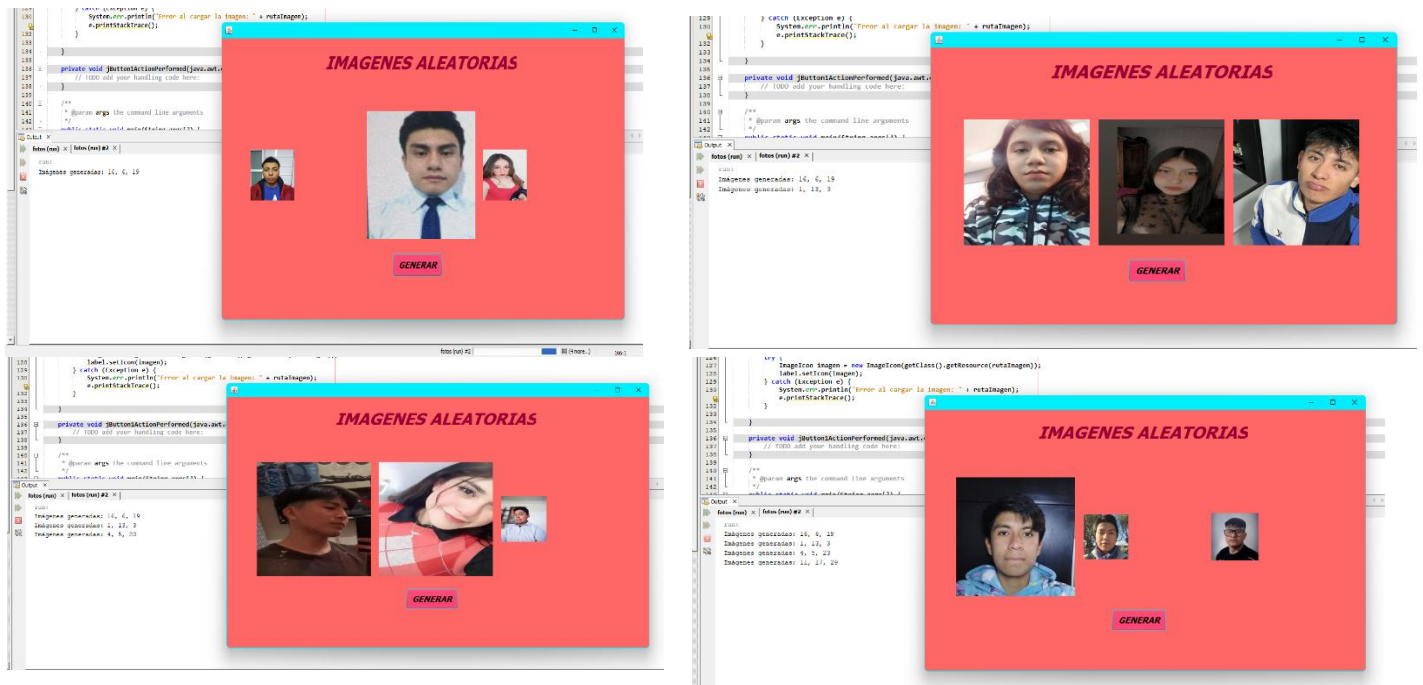
    private javax.swing.JPanel jPanel1;

    // End of variables declaration

}

```

Figura 4. Ejecución del código



Conclusión

En conclusión, el estudio de los números pseudoaleatorios es esencial en el campo de la simulación, ya que permite replicar fenómenos aleatorios de manera controlada. A lo largo de esta unidad, se analizaron diversos métodos, desde los más básicos como Cuadrados Medios hasta los más complejos como los algoritmos Congruenciales, cada uno con sus propias ventajas y aplicaciones prácticas.

Se observó que, aunque estos algoritmos generan números que no son completamente aleatorios, su capacidad para producir secuencias repetibles los hace valiosos en simulaciones computarizadas. Además, se destacó la importancia de seleccionar el método adecuado según las necesidades del problema, considerando factores como la eficiencia y la longitud de la secuencia requerida.

Referencias Bibliográficas

AUTORÍA PROPIA